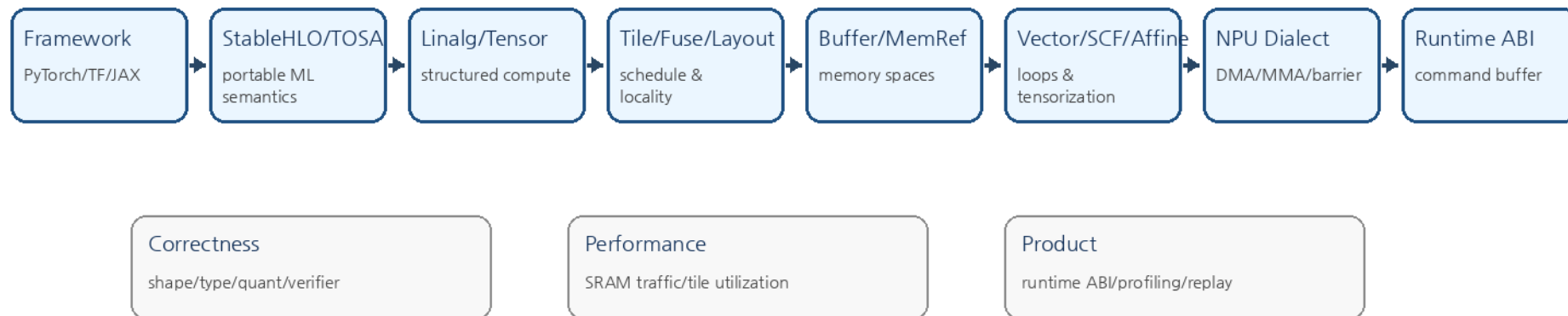


AI and NPU Compiler를 위한 MLIR 20강

From MLIR Basics to Custom Edge NPU Compiler

MLIR-based AI/NPU Compiler Lowering Pipeline



20-Lecture Roadmap (1/2)

주제와 핵심 실습

1	MLIR와 AI/NPU Compiler 큰 그림	NPU compiler lowering pipeline 설계
2	MLIR IR 기본 구조	Operation/Value/Type/Attribute 읽기
3	Region과 Block	NPU kernel/command region pseudo IR
4	mlir-opt, mlir-translate, FileCheck	pass pipeline 실행과 regression test
5	Dialect 시스템	frontend/graph/kernel/runtime dialect 분리
6	StableHLO, TOSA, AI 모델 입력 IR	LayerNorm/GELU primitive decomposition
7	Linalg-on-Tensors	linalg.generic MatMul/Conv 표현
8	SCF/Affine Loop	M/N/K tile loop와 DMA 순서 작성
9	Pass Infrastructure	NPU lowering pass pipeline 초안
10	Rewrite Pattern	MatMul+Add+ReLU fusion pattern 설계

20-Lecture Roadmap (2/2)

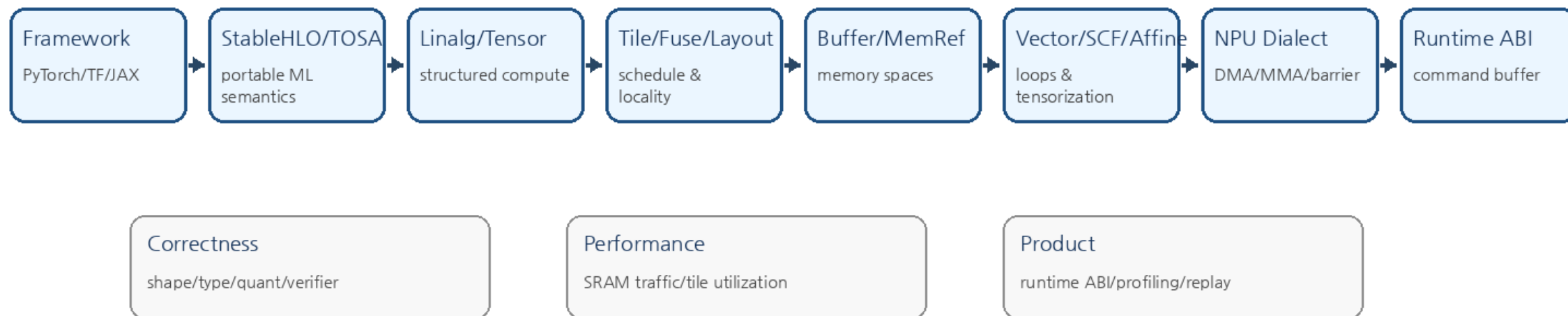
주제와 핵심 실습

11	Dialect Conversion	NPU legal op set과 unsupported op lowering
12	Bufferization	tensor -> memref, alias/lifetime 분석
13	MemRef, Layout, Memory Space	DRAM/SRAM/accumulator memory mapping
14	Vector Dialect, Tensorization	vector.contract -> NPU matmul mapping
15	Transform Dialect	tile/fuse/vectorize schedule IR
16	Quantization	INT8/INT4 requant, accumulator width
17	Fusion, Layout Propagation	Transformer MLP fusion 후보 분석
18	Custom NPU Dialect	npu.matmul/npu.dma/npu.barrier ODS 설계
19	Runtime/ABI/Codegen	command buffer와 tensor descriptor ABI
20	Capstone	Mini NPU Compiler Design Spec 작성

End-to-End MLIR Lowering Pipeline

NPU compiler abstraction ladder

MLIR-based AI/NPU Compiler Lowering Pipeline



Lecture 1: MLIR와 AI/NPU Compiler 큰 그림

NPU compiler lowering pipeline 설계

01

Core Concepts

- Multi-level IR와 dialect 기반 compiler stack
- AI model IR에서 runtime command buffer까지의 lowering ladder
- NPU 병목: SRAM capacity, DRAM bandwidth, DMA overlap, quantization
- Compiler artifact: IR, pass pipeline, kernel command, profiling metadata

NPU Architect Note

- NPU compiler는 TOPS보다 data movement를 먼저 통제해야 한다. 초기 pipeline 설계에서 shape/layout/quant/memory-space 정보를 어느 단계까지 보존할지 결정한다.

Lecture 1: Lab and IR

NPU compiler lowering pipeline 설계

01

Hands-on Tasks

- StableHLO/TOSA -> Linalg -> MemRef/Vector
-> NPU dialect -> Runtime ABI pipeline을
그린다.
- 각 단계의 입력/출력 dialect와 보존 metadata를 적는다.
- unsupported op가 나왔을 때 decomposition, CPU
fallback, custom kernel 중 하나를 고른다.

Representative IR / Pseudo Code

```
Framework Model
-> StableHLO/TOSA
-> Linalg-on-Tensors
-> Tile/Fuse/Layout
-> Bufferization/MemRef
-> Vector/Affine/SCF
-> Custom NPU Dialect
-> Command Buffer + Runtime ABI
```

Lecture 2: MLIR IR 기본 구조

Operation/Value/Type/Attribute 읽기

02

Core Concepts

- Operation은 MLIR의 기본 단위이며 operands/results/attributes/regions를 가진다.
- SSA Value는 op result 또는 block argument로 표현된다.
- Type은 value의 compile-time shape/dtype/layout semantic을 담는다.
- Attribute는 stride, padding, quant scale처럼 static metadata 표현에 적합하다.

NPU Architect Note

- NPU backend에서는 kernel parameter를 attribute로, tensor dataflow를 SSA value로 분리해 verifier와 lowering pattern이 쉽게 읽게 한다.

Hands-on Tasks

- 간단한 StableHLO add/relu IR에서 op, operand, result, type을 표시한다.
- attribute와 type에 들어갈 정보를 분리한다.
- debugging을 위해 location을 남기는 이유를 설명한다.

Representative IR / Pseudo Code

```
func.func @add_relu(%a: tensor<4xf32>, %b: tensor<4xf32>)\n-> tensor<4xf32> {\n  %0 = stablehlo.add %a, %b : tensor<4xf32>\n  %z = stablehlo.constant dense<0.0> : tensor<4xf32>\n  %1 = stablehlo.maximum %0, %z : tensor<4xf32>\n  return %1 : tensor<4xf32>\n}
```


Lecture 3: Region과 Block

NPU kernel/command region pseudo IR

03

Core Concepts

- Region은 nested IR container이고, block은 argument와 terminator를 가진다.
- scf.for, scf.if, linalg.generic, transform op는 region 구조를 사용한다.
- Dominance와 region isolation은 pass correctness의 핵심이다.
- Region은 kernel body나 command sequence를 구조적으로 표현하는 데 유용하다.

NPU Architect Note

- npu.kernel region 안에 DMA, compute, barrier, store를 넣으면 dependency와 scheduling constraint를 IR verifier로 검사할 수 있다.

Lecture 3: Lab and IR

NPU kernel/command region pseudo IR

03

Hands-on Tasks

- scf.for 3중 loop의 region/block 구조를 표시한다.
- pseudo npu.kernel region에 dma_load, matmul, barrier, dma_store를 배치한다.
- terminator가 필요한 region과 implicit terminator region을 구분한다.

Representative IR / Pseudo Code

```
npu.kernel @tile_mma {  
  npu.dma_load %A_dram, %A_sram  
  npu.dma_load %B_dram, %B_sram  
  npu.barrier  
  %acc = npu.matmul %A_sram, %B_sram, %acc0  
  npu.dma_store %acc, %C_dram  
  npu.return  
}
```

Lecture 4: mlir-opt, mlir-translate, FileCheck

04

pass pipeline 실행과 regression test

Core Concepts

- mlir-opt는 pass pipeline 실행과 test 작성의 기본 도구이다.
- mlir-translate는 MLIR와 외부 표현 간 변환을 담당한다.
- FileCheck는 IR 변환 결과를 안정적으로 검증한다.
- split-input-file, verify-diagnostics, CHECK-DAG를 실전에서 자주 쓴다.

NPU Architect Note

- NPU compiler는 pass 수가 많아질수록 regression test 없이는 유지보수가 어렵다. fusion/legalization/bufferization/codegen마다 FileCheck를 둔다.

Lecture 4: Lab and IR

pass pipeline 실행과 regression test

04

Hands-on Tasks

- canonicalize/cse pipeline을 실행한다.
- CHECK-LABEL, CHECK, CHECK-SAME, CHECK-DAG를 사용해 test를 쓴다.
- negative verifier test를 하나 작성한다.

Representative IR / Pseudo Code

```
// RUN: mlir-opt %s -canonicalize -cse | FileCheck %s
// CHECK-LABEL: func.func @matmul
// CHECK: linalg.matmul
```

Lecture 5: Dialect 시스템

frontend/graph/kernel/runtime dialect 분리

05

Core Concepts

- Dialect는 op/type/attribute의 namespace이자 abstraction boundary이다.
- ODS/TableGen은 op definition, verifier, parser/printer 생성을 돕는다.
- Trait와 Interface는 target-independent pass 작성을 가능하게 한다.
- Dialect conversion은 source dialect를 target dialect legal set으로 바꾸는 과정이다.

NPU Architect Note

- 하나의 npu dialect에 graph, kernel, runtime 의미를 모두 넣기보다 level을 나누면 legalization과 debug가 쉬워진다.

Lecture 5: Lab and IR

frontend/graph/kernel/runtime dialect 분리

05

Hands-on Tasks

- frontend, graph, kernel, runtime dialect의 책임을 표로 나눈다.
- npu.matmul, npu.dma, npu.barrier의 operands/results/attributes를 정의한다.
- 각 op에 필요한 verifier constraint를 적는다.

Representative IR / Pseudo Code

```
npu.matmul {tile_m = 16, tile_n = 16, tile_k = 64,  
acc_type = i32}  
  (%a, %b, %acc) : memref<16x64xi8, 1>, memref<64x16xi8,  
1>, memref<16x16xi32, 2>
```

Lecture 6: StableHLO, TOSA, AI 모델 입력 IR

06

LayerNorm/GELU primitive decomposition

Core Concepts

- StableHLO는 ML framework와 compiler 사이 portable HLO semantic을 제공한다.
- TOSA는 DNN에서 많이 쓰는 tensor-level op set과 quantization detail을 제공한다.
- Frontend IR은 모델 의미 보존에 강하지만 NPU schedule에는 아직 이른다.
- Composite op decomposition은 numerical contract와 fusion 가능성을 함께 고려해야 한다.

NPU Architect Note

- 지원 op set이 제한된 NPU에서는 StableHLO/TOSA 단계에서 unsupported op를 빨리 찾아 decomposition 또는 fallback을 결정해야 한다.

Hands-on Tasks

- GELU를 erf 기반 primitive sequence로 표현한다.
- LayerNorm을 reduce/add/mul/rsqrt/add/mul로 분해한다.
- decomposition 후 fusion 가능한 subgraph를 표시한다.

Representative IR / Pseudo Code

```
LayerNorm(x) = (x - mean(x)) * rsqrt(var(x) + eps) *  
gamma + beta  
GELU(x) ~= 0.5 * x * (1 + erf(x / sqrt(2)))
```


Lecture 7: Linalg-on-Tensors

linalg.generic MatMul/Conv 표현

07

Core Concepts

- Linalg는 structured op, indexing map, iterator type으로 tensor compute를 표현한다.
- Tensor semantic은 mutation이 없어 fusion/tiling reasoning이 쉽다.
- linalg.generic은 MatMul/Conv/elementwise/reduction을 통합 표현할 수 있다.
- Linalg 단계는 tile/fuse/vectorize의 중심이다.

NPU Architect Note

- Linalg 단계에서 producer-consumer fusion과 tile 후보를 정하면 SRAM round-trip을 크게 줄일 수 있다.

Lecture 7: Lab and IR

linalg.generic MatMul/Conv 표현

07

Hands-on Tasks

- linalg.matmul과 linalg.generic matmul을 비교한다.
- Conv2D NHWC/HWCF indexing map을 손으로 해석한다.
- output tensor shape를 계산한다.

Representative IR / Pseudo Code

```
%0 = linalg.matmul ins(%A, %B : tensor<128x64xf32>,
tensor<64x256xf32>)
                                outs(%C : tensor<128x256xf32>) ->
tensor<128x256xf32>
```

Lecture 8: SCF/Affine Loop

M/N/K tile loop와 DMA 순서 작성

08

Core Concepts

- SCF는 structured loop/control flow를 표현한다.
- Affine은 affine map 기반 분석과 loop transform에 유리하다.
- Loop order는 activation/weight/accumulator reuse 정책을 결정한다.
- DMA와 compute overlap은 loop scheduling 문제로 표현할 수 있다.

NPU Architect Note

- M/N/K tile loop 순서가 SRAM footprint, accumulator lifetime, DMA burst efficiency, systolic array utilization을 좌우한다.

Lecture 8: Lab and IR

M/N/K tile loop와 DMA 순서 작성

08

Hands-on Tasks

- M/N/K 3중 tile loop를 작성한다.
- double buffering을 위한 load/compute/store 순서를 배치한다.
- tile size가 SRAM capacity를 넘지 않는지 계산한다.

Representative IR / Pseudo Code

```
scf.for %m = %c0 to %M step %TM {  
  scf.for %n = %c0 to %N step %TN {  
    scf.for %k = %c0 to %K step %TK {  
      // dma A/B tile, compute, accumulate  
    }  
  }  
}
```

Lecture 9: Pass Infrastructure

NPU lowering pass pipeline 초안

09

Core Concepts

- Pass는 특정 Operation scope에서 IR을 분석/변환한다.
- Analysis preservation과 invalidation을 이해해야 pipeline이 안정적이다.
- Canonicalize/CSE는 대부분의 lowering 단계 사이에 들어간다.
- Pass option, statistic, diagnostic은 실전 compiler 개발에 필수이다.

NPU Architect Note

- NPU pipeline은 legalize, fuse, tile, bufferize, memory plan, schedule, codegen이 서로 의존하므로 pass boundary와 invariant를 명확히 해야 한다.

Lecture 9: Lab and IR

NPU lowering pass pipeline 초안

09

Hands-on Tasks

- 10개 내외 pass로 lowering pipeline을 설계한다.
- 각 pass의 input/output dialect와 invariant를 적는다.
- 실패 diagnostic message를 설계한다.

Representative IR / Pseudo Code

```
builtin.module(  
  canonicalize, cse,  
  func.func(tosa-to-linalg, npu-fuse, npu-tile),  
  one-shot-bufferize, npu-memory-plan, npu-codegen)
```

Lecture 10: Rewrite Pattern

10

MatMul+Add+ReLU fusion pattern 설계

Core Concepts

- RewritePattern은 local IR transformation을 구현한다.
- Greedy rewrite는 canonicalization과 fusion에서 자주 사용된다.
- Pattern benefit은 rewrite 우선순위를 조정한다.
- DAG matching에서는 use-def chain과 multiple-use 여부가 중요하다.

NPU Architect Note

- Fusion의 목적은 op 수 감소가 아니라 DRAM round-trip 제거와 quantization boundary 최소화이다.

Lecture 10: Lab and IR

10

MatMul+Add+ReLU fusion pattern 설계

Hands-on Tasks

- MatMul -> Add -> ReLU pattern을 찾아 fused op로 바꾸는 조건을 작성한다.
- bias add와 broadcast add를 구분한다.
- fusion하면 안 되는 경우를 세 가지 적는다.

Representative IR / Pseudo Code

```
%0 = linalg.matmul ...  
%1 = arith.addi %0, %bias  
%2 = arith.maxsi %1, %zero  
// -> npu.matmul_bias_relu
```


Lecture 11: Dialect Conversion

NPU legal op set과 unsupported op lowering

11

Core Concepts

- ConversionTarget은 legal/illegal/dynamic legal op를 정의한다.
- TypeConverter는 tensor/memref/layout/quant type 변환을 담당한다.
- ConversionPattern은 source op를 target dialect로 낮춘다.
- Partial conversion과 full conversion을 구분해야 한다.

NPU Architect Note

- NPU legal op set은 실제 hardware ISA와 runtime ABI의 계약이다. 미지원 op는 decomposition 또는 CPU fallback 정책이 필요하다.

Lecture 11: Lab and IR

NPU legal op set과 unsupported op lowering

11

Hands-on Tasks

- NPU legal op set 12개를 정의한다.
- tosa.matmul, tosa.conv2d, tosa.rescale의 lowering target을 정한다.
- dynamic legality 조건을 작성한다.

Representative IR / Pseudo Code

```
Legal: npu.matmul, npu.conv2d, npu.dma_load,  
npu.dma_store, npu.barrier  
Illegal after legalization: stablehlo.*, tosa.*
```

Lecture 12: Bufferization

tensor -> memref, alias/lifetime 분석

12

Core Concepts

- Tensor semantic은 value-based, memref semantic은 buffer/mutation-based이다.
- One-shot bufferization은 in-place 가능성을 분석한다.
- Destination-passing style은 bufferization 품질을 높인다.
- Alias/lifetime 분석은 memory planning의 출발점이다.

NPU Architect Note

- Bufferization 결과는 DRAM/SRAM/ACC placement와 DMA 계획의 직접 입력이다.

Lecture 12: Lab and IR

12

tensor -> memref, alias/lifetime 분석

Hands-on Tasks

- tensor.empty + linalg.matmul을 memref 기반 실행으로 해석한다.
- in-place 가능한 result와 copy가 필요한 result를 표시한다.
- activation lifetime interval을 그린다.

Representative IR / Pseudo Code

```
tensor<128x256xi8>  
-> memref<128x256xi8, strided<[256,1], offset: ?>>
```

Lecture 13: MemRef, Layout, Memory Space

13

DRAM/SRAM/accumulator memory mapping

Core Concepts

- MemRef는 shape, element type, layout map, memory space를 가진다.
- Layout map은 physical address와 vectorization 품질을 좌우한다.
- Memory space는 DRAM/SRAM/register/accumulator 구분에 사용된다.
- Alignment, bank conflict, stride는 NPU 성능과 직접 연결된다.

NPU Architect Note

- Edge NPU는 작은 batch와 latency가 중요하므로 bank conflict와 DMA burst 효율이 peak TOPS만큼 중요하다.

Lecture 13: Lab and IR

DRAM/SRAM/accumulator memory mapping

13

Hands-on Tasks

- NHWC, NCHW, blocked layout의 address formula를 작성한다.
- memory space 0/1/2를 DRAM/SRAM/ACC로 매핑한다.
- tile layout이 bank conflict를 만드는지 분석한다.

Representative IR / Pseudo Code

```
memref<1x224x224x64xi8, #nhwc, 0> // DRAM
memref<16x16x64xi8, #tile, 1>      // SRAM
memref<16x16xi32, #acc, 2>         // ACC
```

Lecture 14: Vector Dialect, Tensorization

14

vector.contract -> NPU matmul mapping

Core Concepts

- Vector dialect는 target-independent SIMD/tensor operation을 표현한다.
- vector.contract는 matmul/convolution tensorization의 중간 표현으로 유용하다.
- vector.transfer_read/write는 memory와 vector 사이 이동을 표현한다.
- Native NPU MMA shape과 vector shape를 맞추면 lowering이 단순해진다.

NPU Architect Note

- NPU array의 native tile shape을 vector.contract shape로 표현하면 backend mapping과 verifier가 명확해진다.

Lecture 14: Lab and IR

vector.contract -> NPU matmul mapping

14

Hands-on Tasks

- 16x16x64 int8 matmul을 vector.contract로 표현한다.
- transfer_read/write alignment 조건을 적는다.
- contract shape과 systolic array shape을 매핑한다.

Representative IR / Pseudo Code

```
%acc2 = vector.contract {iterator_types =  
  ["parallel","parallel","reduction"]}  
  %va, %vb, %acc : vector<16x64xi8>, vector<64x16xi8>  
  into vector<16x16xi32>
```


Lecture 15: Transform Dialect

tile/fuse/vectorize schedule IR

15

Core Concepts

- Transform dialect는 payload IR을 변환하는 schedule을 IR로 표현한다.
- Computation IR과 transformation recipe를 분리한다.
- Tiling, fusion, vectorization을 script처럼 구성할 수 있다.
- Autotuning/search space 표현과 잘 맞는다.

NPU Architect Note

- Tile size, fusion group, vector shape를 transform IR parameter로 두면 NPU schedule 탐색과 재현이 쉬워진다.

Lecture 15: Lab and IR

tile/fuse/vectorize schedule IR

15

Hands-on Tasks

- linalg.matmul을 match하는 transform sequence를 작성한다.
- tile size [16,16,64]를 적용한다.
- producer fusion과 vectorization 순서를 바꿔본다.

Representative IR / Pseudo Code

```
transform.sequence failures(propagate) {  
  %m = transform.structured.match ops{["linalg.matmul"]}  
  in %root  
    %tiled, %loops = transform.structured.tile_using_for %m  
    [16, 16, 64]  
}
```

Lecture 16: Quantization

INT8/INT4 requant, accumulator width

16

Core Concepts

- Quantization은 scale, zero-point, accumulator type, rounding/saturation policy의 조합이다.
- Per-tensor/per-channel quantization은 accuracy와 hardware cost trade-off를 만든다.
- Requantization은 int32 accumulator를 int8/int4 output으로 되돌리는 과정이다.
- Calibration/QAT/PTQ metadata가 IR에 안전하게 전달되어야 한다.

NPU Architect Note

- INT8/INT4 TOPS를 latency 이득으로 바꾸려면 dequant/requant가 fusion boundary 밖으로 새지 않아야 한다.

Lecture 16: Lab and IR

INT8/INT4 requant, accumulator width

16

Hands-on Tasks

- int8 matmul의 int32 accumulator 범위를 계산한다.
- requant pseudo IR을 작성한다.
- per-channel scale이 layout에 미치는 영향을 분석한다.

Representative IR / Pseudo Code

```
acc_i32 = sum((a_i8 - zp_a) * (b_i8 - zp_b))  
out_i8 = clamp(round(acc_i32 * M >> S) + zp_out, -128,  
127)
```

Lecture 17: Fusion, Layout Propagation

17

Transformer MLP fusion 후보 분석

Core Concepts

- Fusion은 locality, launch overhead, memory traffic을 줄인다.
- Layout propagation은 transpose/reshape/copy를 줄이는 핵심 기법이다.
- Fusion legality는 shape, side effect, quantization, memory pressure로 제한된다.
- Cost model 없이는 과도한 fusion이 SRAM spill을 만들 수 있다.

NPU Architect Note

- Transformer MLP에서 MatMul+Bias+GELU를 SRAM에 머물게 할지, 두 번째 MatMul까지 확장할지가 latency를 좌우한다.

Lecture 17: Lab and IR

Transformer MLP fusion 후보 분석

17

Hands-on Tasks

- MLP block graph를 그리고 fusion 후보를 표시한다.
- 각 후보의 SRAM footprint를 계산한다.
- layout propagation으로 제거 가능한 transpose를 찾는다.

Representative IR / Pseudo Code

```
X -> MatMul(W1) -> Bias -> GELU -> MatMul(W2) -> Bias  
Candidate A: MatMul + Bias + GELU  
Candidate B: Full MLP fusion if SRAM allows
```

Lecture 18: Custom NPU Dialect

18

`npu.matmul/npu.dma/npu.barrier` ODS 설계

Core Concepts

- Custom dialect는 hardware contract를 명시하는 지점이다.
- ODS는 syntax, verifier, trait, interface 정의를 구조화한다.
- Verifier는 tile size, dtype, memory space constraint를 빠르게 잡아야 한다.
- Dialect 문서와 tests는 backend bring-up 속도를 좌우한다.

NPU Architect Note

- npu dialect는 ISA를 그대로 노출하기보다 compiler optimization이 가능한 semantic richness를 남기는 것이 좋다.

Lecture 18: Lab and IR

npu.matmul/npu.dma/npu.barrier ODS 설계

18

Hands-on Tasks

- npu.matmul, npu.dma_load, npu.dma_store, npu.barrier, npu.launch op를 정의한다.
- tile shape/dtype/memory space verifier 조건을 적는다.
- ODS TableGen skeleton을 작성한다.

Representative IR / Pseudo Code

```
def NPU_MatMulOp : NPU_Op<"matmul", [Pure]> {  
  let arguments = (ins NPU_SRAM:$a, NPU_SRAM:$b,  
    NPU_ACC:$acc);  
  let results = (outs NPU_ACC:$result);  
}
```


Lecture 19: Runtime/ABI/Codegen

command buffer와 tensor descriptor ABI

19

Core Concepts

- Codegen은 IR을 executable artifact와 runtime metadata로 바꾼다.
- Runtime ABI는 tensor descriptor, command buffer, sync, error model을 정의한다.
- Host/device memory ownership과 cache coherency가 edge deployment에서 중요하다.
- Profiling metadata는 compiler feedback loop의 입력이다.

NPU Architect Note

- Autonomous driving/robotics는 tail latency와 determinism이 중요하므로 runtime ABI에서 dynamic allocation과 unpredictable sync를 줄여야 한다.

Lecture 19: Lab and IR

command buffer와 tensor descriptor ABI

19

Hands-on Tasks

- tensor descriptor 구조체를 설계한다.
- command buffer packet format을 정의한다.
- submit/sync/profile runtime API를 작성한다.

Representative IR / Pseudo Code

```
struct TensorDesc { uint64_t addr; uint32_t rank; int64_t  
shape[6]; int64_t stride[6]; uint32_t dtype; uint32_t  
layout; };  
struct Cmd { uint16_t opcode; uint16_t flags; uint32_t  
bytes; uint64_t payload; };
```

Lecture 20: Capstone

Mini NPU Compiler Design Spec 작성

20

Core Concepts

- 입력 IR, legal op set, pass pipeline, memory model, codegen ABI를 하나의 설계 문서로 통합한다.
- 정확성은 verifier, FileCheck, numerical tolerance로 검증한다.
- 성능은 roofline, SRAM traffic, tile utilization, bandwidth로 추정한다.
- 제품화에는 debug, profiling, replay 기능이 필요하다.

NPU Architect Note

- 최종 산출물은 작은 compiler라도 실제 NPU bring-up 회의에서 리뷰 가능한 설계 명세여야 한다.

Lecture 20: Lab and IR

Mini NPU Compiler Design Spec 작성

20

Hands-on Tasks

- Transformer MLP 또는 Conv block 하나를 end-to-end lowering한다.
- Mini NPU Compiler Design Spec 10-15페이지를 작성한다.
- 성능/메모리/정확성 리스크와 mitigation을 정리한다.

Representative IR / Pseudo Code

Capstone checklist:

- ☐ Supported ops and dtypes
- ☐ MLIR pipeline
- ☐ SRAM/DRAM memory plan
- ☐ NPU dialect ops
- ☐ Runtime ABI
- ☐ Tests and profiling hooks

References

Official baseline documents

- MLIR Language Reference: <https://mlir.llvm.org/docs/LangRef/>
- MLIR Dialects: <https://mlir.llvm.org/docs/Dialects/>
- MLIR Pass Infrastructure: <https://mlir.llvm.org/docs/PassManagement/>
- MLIR Bufferization: <https://mlir.llvm.org/docs/Bufferization/>
- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>
- MLIR Transform Dialect: <https://mlir.llvm.org/docs/Dialects/Transform/>
- StableHLO Specification: <https://openxla.org/stablehlo/spec>
- TOSA Specification: https://www.mlplatform.org/tosa/tosa_spec.html
- MLIR TOSA Dialect: <https://mlir.llvm.org/docs/Dialects/TOSA/>